

Tiny Vision-Language Model

Rayen Touil | RLS Entrance Challenge | Level 1

1. The task

The model sees a 64 x 64 image of one or two shapes and writes a description, one letter at a time. For example, a large red circle becomes **largeredcircle**. The main question is whether it uses the image or only learns how the words are spelled.

2. Architecture

RGB image	(B, 3, 64, 64)
3 convolutions + ReLU, then average pooling	(B, 128, 4, 4)
Flatten the grid + linear adapter	(B, 16, 128)
Visual tokens + letters + learned positions	(B, 16 + T, 128)
2 decoder blocks, 4 attention heads	(B, 16 + T, 128)
Last visual token + letters -> output head	(B, T + 1, 27)

Figure 1. B is the batch size and T is the number of input letters. The last visual token predicts the first letter.

The model has **521,307 parameters**. The 4 x 4 grid keeps spatial information with only 16 visual tokens. Each token has 128 values; each of the four heads uses 32. The MLP expands 128 to 512, then returns to 128. This is a small baseline, not a claim that these settings are the best.

3. Implementation

Attention is written by hand: $\text{softmax}(\mathbf{QK}^T / \sqrt{32} + \text{mask})\mathbf{V}$. Visual tokens can read each other, but cannot read the letters. A letter can read the image and its current prefix. Learned positions give the tokens an order. Each decoder block uses LayerNorm before attention and the MLP, with residual additions.

The vocabulary has 26 letters and eos. No start token is needed. During training, the correct previous letters are supplied (teacher forcing). Padded targets use -100 and are ignored by the loss. During generation, the model uses its own predictions and stops at eos or 45 letters.

The attention is checked against PyTorch's reference at about 1e-5 tolerance in float32. Other tests check gradients, future-token masking, target alignment, padding and generation. No pretrained model or built-in Transformer is used.

4. Training setup

Setting	Value
Data	Official generator, unchanged; seed 42. Train 20,000; val, test and heldout 2,000 each.
Input	RGB pixels stored as uint8, converted to float32 and divided by 255.
Training	Batch 64; 10 epochs; seed 42.
Optimizer	AdamW; learning rate 0.001; weight decay 0.01; gradient norm clipped to 1.
Checkpoint	Lowest validation loss. The test split is not used to select the model.
Hardware	NVIDIA GeForce GTX 1650; Intel(R) Core(TM) i5-10500H CPU @ 2.50GHz; 4 CPU threads.
Software	Python 3.12.14; PyTorch 2.8.0+cu126; CUDA 12.6; Windows 11.

5. Experiments

E0 - Overfit one batch. The prediction before running was that the model should learn a fixed batch of 16 examples and reach a loss below 0.02. After 600 steps, the loss was **0.000372 nats/token** and exact match was **100.00%**. This checks that learning works on that batch; it does not measure performance on new images.

E1 - Blind baseline. The prediction was that real images would give better word and attribute accuracy than zero images. Both models use the same settings, seed, initialization and batch order. The blind model receives zero images in both training and evaluation. Only one seed was run for each setup.

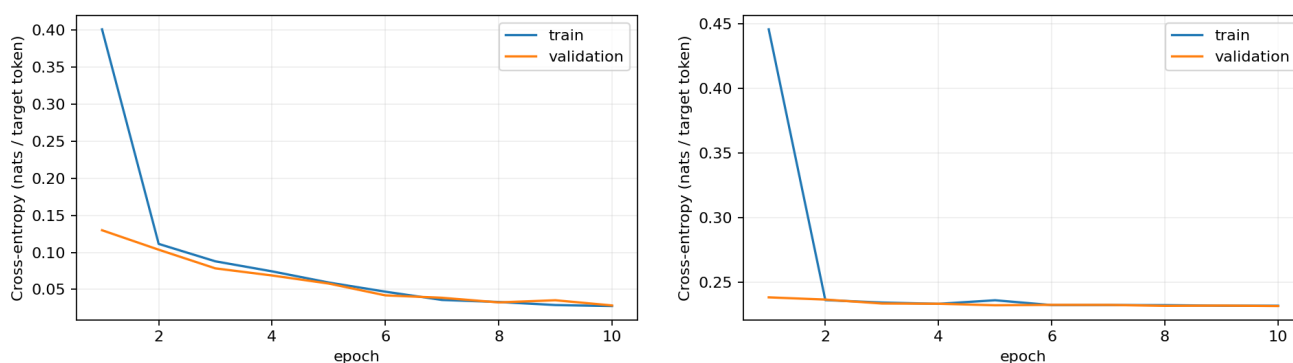


Figure 2. Training and validation loss on NVIDIA GeForce GTX 1650. Left: visual model. Right: blind model. The blind loss stops improving much earlier. Note the different vertical scales.

Loss is averaged over real target tokens, with padding excluded. CSV files keep the values behind the plots. Checkpoints save model weights, optimizer state and progress so training can resume after a completed epoch.

6. Results

Metric	Visual / test	Blind / test	Visual / heldout
Exact match	68.05%	1.70%	0.00%
Size	84.80%	33.28%	80.14%
Color	74.83%	20.10%	75.80%
Shape	73.81%	15.30%	23.32%
Relation	44.22%	0.00%	44.63%
Object count	99.90%	49.00%	97.60%
Letters, free generation	68.99%	16.17%	47.29%
Letters, true prefix	98.52%	87.34%	89.73%

Table 1. Results on NVIDIA GeForce GTX 1650, one training seed per setup. No standard deviation across seeds is available. Test has 2,000 images, 3,020 objects and 1,020 relations; heldout has 2,000 images, 2,950 objects and 950 relations.

The parser keeps correct parts of a broken word. For **largerredcirle**, size and color are correct, but shape is wrong. It does not fix spelling. Attributes are scored at their reference object position; relations are scored only for two-object scenes. Object count is inferred from size-word boundaries, so malformed outputs can confuse it.

Free-generation letter accuracy compares matching positions and divides by the longer word length. True-prefix accuracy supplies the correct earlier letters and excludes eos and padding. These are different tests.

7. What the results show

The visual model reaches **68.05%** exact match, against **1.70%** for the blind model. This supports the use of image information in this run. However, the blind model gets **87.34%** of letters right when given the true prefix. Much of the spelling is predictable without seeing the image.

Exact match is 99.49% on 980 one-object scenes and 37.84% on 1020 two-object scenes. Of 639 strict errors, 381 name the same objects in reverse order with the inverse relation: A below B becomes B above A. The generator does not set a fixed object order. These remain errors in the official metric.

The remaining 258 errors show that word order does not explain everything. Heldout exact match is 0.00%; shape accuracy falls to 23.32%. For example, **largeyellowcross** becomes **largeyellowcircle**. A possible explanation is that the model learned common color-shape pairs, but did not learn to combine them freely. This needs another experiment to test.

8. CPU and GPU timing

Batch	CPU images/s	GPU images/s
1	61.88 +/- 7.62	98.47 +/- 5.52
16	422.40 +/- 30.51	1434.32 +/- 116.73
64	507.68 +/- 24.40	4414.42 +/- 9.09
256	495.07 +/- 40.45	4455.75 +/- 7.36

Table 2. Mean +/- sample standard deviation across five repeats of ten training steps, after five warm-up steps. Intel i5-10500H, four threads; NVIDIA GTX 1650. Float32, 16 visual tokens plus 45 input letters.

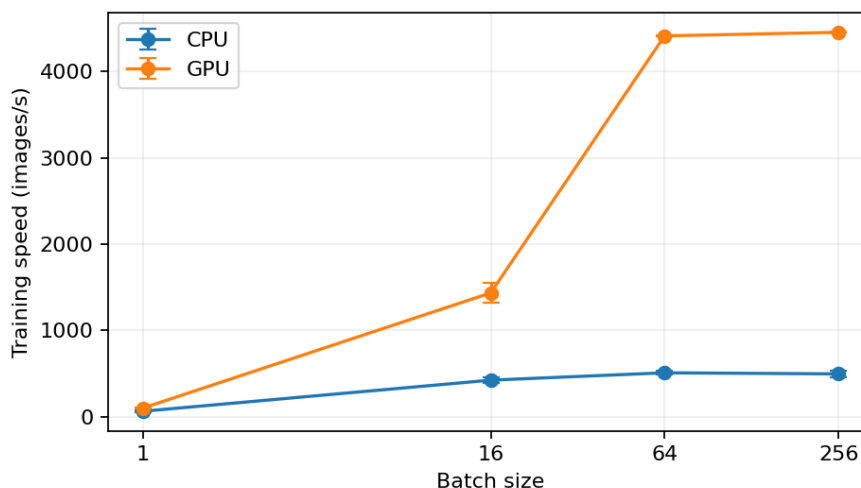


Figure 3. The GPU is faster at all four tested batch sizes. Its advantage is smaller at batch 1, and throughput changes little between 64 and 256.

The timer includes forward, loss, backward, gradient clipping and AdamW. It excludes data loading, transfers and file writes: synthetic tensors are already on the device. Warm-up removes initial setup costs. CUDA synchronization before and after timing waits for the asynchronous GPU work. S1 ran without another training job.

Larger batches can spread fixed costs across more images and provide more parallel work. No CPU/GPU crossover was observed here. A profiler would be needed to explain the exact bottleneck. Attention scores use $B \times H \times T$ squared values, so longer sequences also cost memory.

9. Limits and next step

This is one small model with one seed on synthetic images. The parser and object-order ambiguity affect the metrics. No detailed profiling, memory study or accuracy-tuning search was run. The next useful step is to test why unseen color-shape pairs fail. The heldout check is exploratory; this report does not claim Level 2.

References

RLS: github.com/RLS-ResearchLab/RLS-Entrance-Challenge, template c21f9da.
Vaswani et al., Attention Is All You Need (2017): arxiv.org/abs/1706.03762.
PyTorch 2.8: `scaled_dot_product_attention` and `CrossEntropyLoss`; direct links in SOURCE.md.
Learning and coding assistance are described in AI_USAGE.md.